

GUIDE — TODO APP

C# ASP.NET Core + React

Schemas d'architecture · Guide pas a pas

15 taches pour construire un projet full-stack de A a Z

C# .NET 8

React 18

SQLite

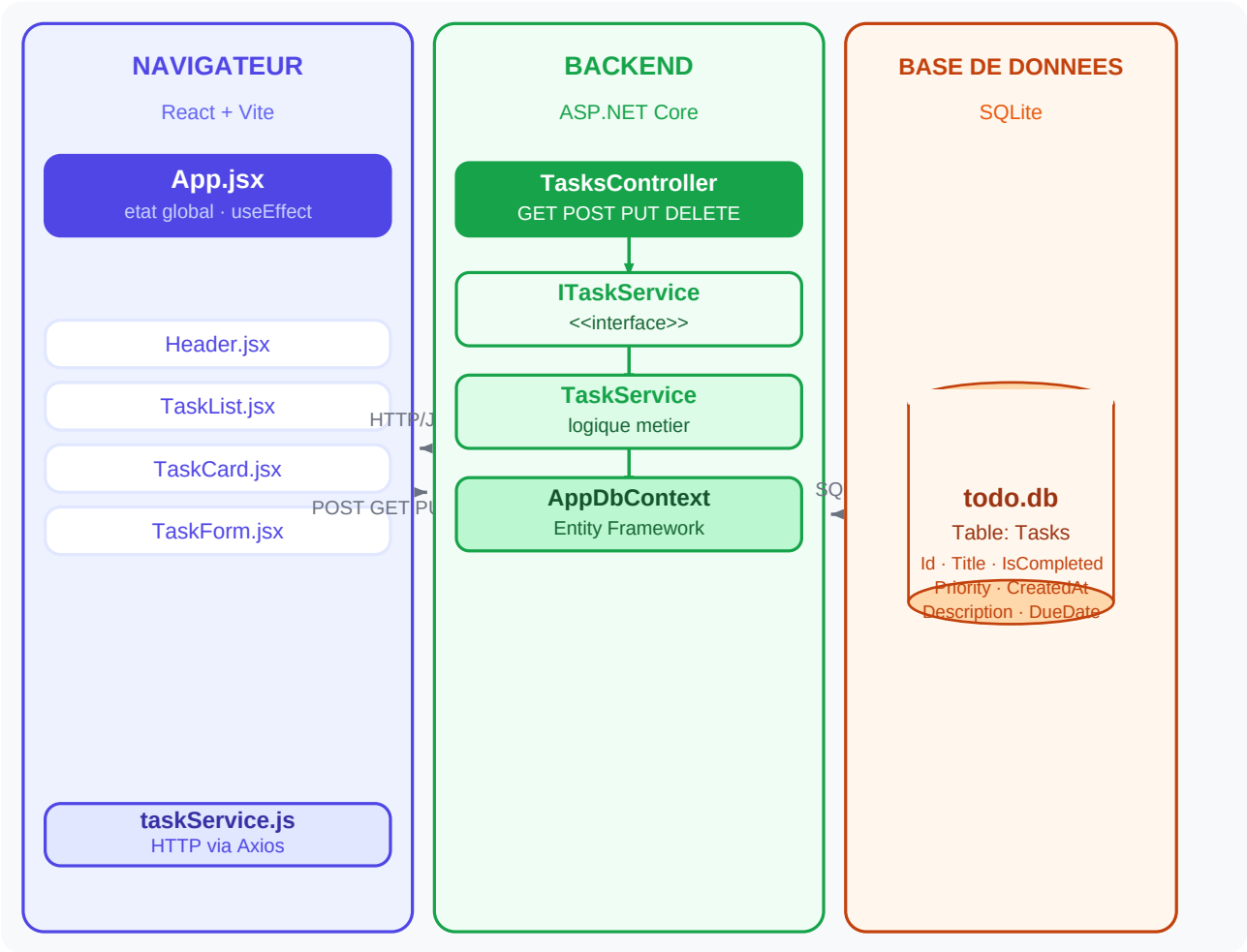
Docker

Git

Ce guide te donne les **etapes concretes** pour construire le projet de A a Z par toi-meme, en comprenant chaque concept. Les schemas d'architecture en pages suivantes te donnent une vue d'ensemble avant de commencer a coder.

Schema 1 — Architecture globale

Vue d'ensemble des trois couches du projet et de leurs interactions.



Frontend React ↔ Backend ASP.NET Core ↔ Base de données SQLite

Composant	Role	Technologie
Frontend	Interface utilisateur, etat, appels HTTP	React 18 + Vite
Backend	API REST, logique metier, validation	C# ASP.NET Core 8
Base de donnees	Stockage persistant des taches	SQLite + EF Core

Schema 2 — Flux d'une requete (POST /api/tasks)

Chemin complet d'un clic "Créer une tâche" jusqu'à la base de données et retour.

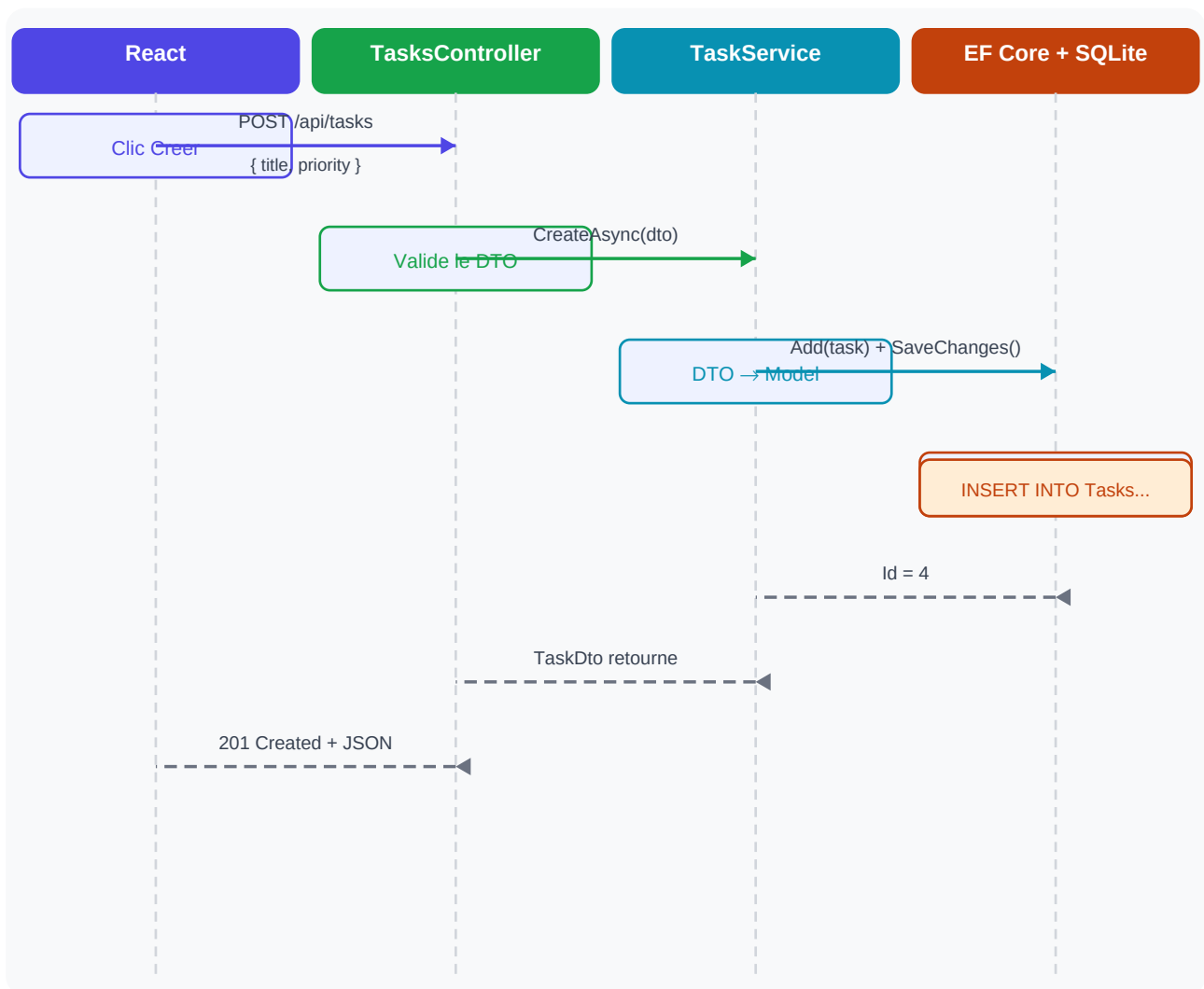


Diagramme de séquence simplifié — aller (flèches pleines) · retour (pointilles)

■ Pourquoi autant de couches ?

Chaque couche a une responsabilité unique : le Contrôleur gère HTTP, le Service gère la logique, le DbContext gère la BDD. Si tu changes la BDD (SQLite → PostgreSQL), seul ApplicationDbContext change. C'est le principe de séparation des responsabilités (SRP — SOLID).

Schema 3 — Structure des fichiers

Arborescence complete du projet avec le role de chaque fichier.

■ todo-app/	
■ .gitignore	<i>ignore bin/, node_modules/, *.db</i>
■ docker-compose.yml	<i>orchestre backend + frontend</i>
■ backend/	
■ Dockerfile	<i>build multi-stage .NET</i>
■ TodoApp.Api/	
■ Program.cs	<i>config DI, CORS, Swagger, BDD</i>
■ TodoApp.Api.csproj	<i>packages NuGet</i>
■ Models/	<i>TodoTask.cs — entite BDD</i>
■ DTOs/	<i>CreateTaskDto · UpdateTaskDto · TaskDto</i>
■ Data/	<i>AppDbContext.cs — EF Core</i>
■ Services/	<i>ITaskService.cs + TaskService.cs</i>
■ Controllers/	<i>TasksController.cs — 5 endpoints</i>
■ Migrations/	<i>genere par EF Core</i>
■ frontend/	
■ Dockerfile + nginx.conf	<i>build React → Nginx</i>
■ package.json	<i>React, Vite, Axios</i>
■ vite.config.js	<i>proxy /api → localhost:5000</i>
■ src/	
■ App.jsx	<i>etat global · handlers</i>
■ components/	<i>Header · TaskList · TaskCard · TaskForm</i>
■ services/	<i>taskService.js — HTTP centralise</i>
■ styles/	<i>App.css</i>

Les zones colorees indiquent les fichiers les plus importants a creer en premier.

0. Ce qu'il faut installer

Outil	Version	Lien
NET SDK 8	8.0+	dotnet.microsoft.com/download
Node.js	18+	nodejs.org
Git	toute	git-scm.com
VS Code	toute	code.visualstudio.com
Docker Desktop	optionnel	docker.com/products/docker-desktop

Extensions VS Code a installer :

- C# Dev Kit (Microsoft)
- ESLint
- Prettier

Vérification en terminal :

```
dotnet --version # doit afficher 8.x.x
node --version # doit afficher v18.x ou +
git --version
```

1. Setup Git

```
mkdir todo-app
cd todo-app
git init
```

.gitignore a creer a la racine :

```
# C# / .NET
bin/
obj/
*.db
*.db-shm
*.db-wal

# React / Node
node_modules/
dist/
.env

# OS
.DS_Store
Thumbs.db
```

Convention de commits (a respecter tout au long du projet) :

```
git commit -m "feat: ajouter le modele TodoTask"
git commit -m "feat: implémenter le TaskService"
git commit -m "fix: corriger la validation du DTO"
git commit -m "docs: ajouter commentaires"
```

■ Conventional Commits

Cette convention (feat/fix/docs/refactor) est utilisée dans la quasi-totalité des projets open-source et en entreprise. Elle permet de générer des changelogs automatiquement et de rendre l'historique Git lisible par toute l'équipe.

2. Backend C# — 8 tâches

1 Créer le projet ASP.NET Core

Commande à exécuter :

```
mkdir backend && cd backend
dotnet new webapi -n TodoApp.Api --no-openapi
cd TodoApp.Api
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet add package Swashbuckle.AspNetCore
```

■ NuGet = npm pour C#

Les packages NuGet sont les bibliothèques tierces de .NET. Ils s'ajoutent dans le fichier .csproj (équivalent du package.json).

Commit : feat: initialiser le projet ASP.NET Core Web API

2 Créer le Model (Models/ToDoTask.cs)

Ce fichier représente une ligne dans la table de BDD.

Propriétés à implémenter :

- **Id** (int) — clé primaire auto-incrémentée
- **Title** (string) — obligatoire
- **Description** (string?) — le ? = peut être null
- **IsCompleted** (bool) — false par défaut
- **Priority** (enum Priority) — Low / Medium / High
- **CreatedAt** (DateTime) — DateTime.UtcNow par défaut
- **DueDate** (DateTime?) — échéance optionnelle

Commit : feat: ajouter le modèle ToDoTask

3 Créer les DTOs (DTOs/TaskDto.cs)

TaskDto — ce que l'API retourne (tous les champs visibles)

CreateTaskDto — ce que le client envoie pour créer

UpdateTaskDto — tous les champs optionnels (avec ?) pour la modification

■ Pourquoi des DTOs ?

Le Model est lié à la BDD. Si tu l'exposes directement, tout changement de schéma BDD casse l'API. Les DTOs séparent les deux couches.

Commit : feat: ajouter les DTOs

4 Configurer Entity Framework (Data/AppDbContext.cs)

- Hériter de **DbContext**
- Constructeur avec **DbContextOptions**
- Propriété **DbSet<ToDoTask> Tasks**

- Surcharger **OnModelCreating** pour ajouter des données de demo (HasData)

■ Entity Framework Core

EF Core est un ORM — il traduit tes opérations C# en SQL automatiquement. Tu n'écris jamais de SQL à la main. Un DbSet = une table BDD.

Commit : feat: configurer AppDbContext

5 Créer l'interface et le service (Services/)

ITaskService.cs — définir le contrat (5 méthodes) :

```
GetAllAsync() → Task<IEnumerable<TaskDto>>
GetByIdAsync() → Task<TaskDto?>
CreateAsync() → Task<TaskDto>
UpdateAsync() → Task<TaskDto?>
DeleteAsync() → Task<bool>
```

TaskService.cs — implémenter chaque méthode avec EF Core

Chaque méthode doit être async et utiliser await pour les accès BDD.

Commit : feat: implémenter ITaskService et TaskService

6 Créer le Controller (Controllers/TasksController.cs)

Decorateurs à utiliser :

```
[ApiController]
[Route("api/[controller]")]
public class TasksController : ControllerBase
```

5 méthodes avec leurs codes HTTP attendus :

- GET /api/tasks → Ok(tasks) — 200
- GET /api/tasks/{id} → Ok(task) / NotFound() — 200 / 404
- POST /api/tasks → CreatedAtAction(...) — 201
- PUT /api/tasks/{id} → Ok(task) / NotFound() — 200 / 404
- DELETE /api/tasks/{id} → NoContent() / NotFound() — 204 / 404

Commit : feat: créer TasksController avec les 5 endpoints

7 Configurer Program.cs

Dans l'ordre, enregistrer :

1. **AddDbContext** avec `UseSqlite("Data Source=todo.db")`
2. **AddScoped<ITaskService, TaskService>()**
3. **AddCors** pour autoriser `http://localhost:5173`
4. **AddControllers()**, **AddSwaggerGen()**

Au démarrage : **db.Database.Migrate()** pour créer la BDD automatiquement.

Commit : feat: configurer Program.cs (DI, CORS, Swagger)

8 Migration et test Swagger


```
dotnet tool install --global dotnet-ef  
dotnet ef migrations add InitialCreate  
dotnet run
```

Ouvrir <http://localhost:5000/swagger> et tester dans l'ordre :

1. POST /api/tasks → creer une tache (verifie 201)
2. GET /api/tasks → verifier qu'elle apparait (verifie 200)
3. PUT /api/tasks/{id} → modifier isCompleted: true (verifie 200)
4. DELETE /api/tasks/{id} → supprimer (verifie 204)
5. GET /api/tasks/{id} inexistant → verifie le 404

Commit : feat: migration initiale + BDD SQLite fonctionnelle

3. Frontend React — 4 taches

9 Créer le projet React (Vite)

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install axios
```

Dans vite.config.js, ajouter le proxy :

```
server: {
  proxy: {
    '/api': { target: 'http://localhost:5000', changeOrigin: true }
  }
}
```

■ Pourquoi un proxy ?

React tourne sur le port 5173, l'API sur 5000. Le navigateur bloquerait les appels cross-origin. Le proxy Vite redirige /api/* vers le backend en développement.

Commit : feat: initialiser React avec Vite

10 Créer taskService.js (src/services/)

Un objet avec 5 fonctions — une par endpoint API :

```
import axios from 'axios'
const BASE = '/api/tasks'

export default {
  getAll: () => axios.get(BASE).then(r => r.data),
  getById: (id) => axios.get(`${BASE}/${id}`).then(r => r.data),
  create: (data) => axios.post(BASE, data).then(r => r.data),
  update: (id, data) => axios.put(`${BASE}/${id}`, data).then(r => r.data),
  delete: (id) => axios.delete(`${BASE}/${id}`)
}
```

Commit : feat: creer taskService.js

11 Créer les composants (src/components/)

Ordre recommande — du plus simple au plus complexe :

1. **Header.jsx** — pas d'état, reçoit totalTasks et completedTasks en props
2. **TaskCard.jsx** — affiche une tâche, reçoit task/onToggle/onEdit/onDelete
3. **TaskForm.jsx** — useState pour chaque champ, gère création et édition
4. **TaskList.jsx** — useState pour filtre et recherche, filtre côté client

■ Props vs State

State = données propres au composant, modifiables (useState). Props = données venant du parent, en lecture seule. Règle : l'état vit dans le plus petit ancêtre commun à tous les composants qui l'utilisent.

Commit : feat: creer les composants React

12 Assembler App.jsx + tester l'integration

App.jsx gere l'etat global et orchestre tout :

```
// Charger les taches au montage
useEffect(() => { taskService.getAll().then(setTasks) }, [])

// Creer ou modifier selon editingTask
const handleSubmit = async (data) => {
  if (editingTask) {
    const updated = await taskService.update(editingTask.id, data)
    setTasks(prev => prev.map(t => t.id === updated.id ? updated : t))
  } else {
    const created = await taskService.create(data)
    setTasks(prev => [created, ...prev])
  }
}
```

Lancer les deux serveurs en meme temps :

```
# Terminal 1
cd backend/ToDoApp.Api && dotnet run

# Terminal 2
cd frontend && npm run dev

# Ouvrir http://localhost:5173
```

Commit : feat: assembler App.jsx — integration complete

4. Docker — 3 taches (optionnelles)

13 Dockerfile backend (backend/Dockerfile)

Structure en 2 etapes (multi-stage build) :

```
# Etape 1 : SDK (~700MB) pour compiler
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /app
COPY TodoApp.Api/ToDoApp.Api.csproj ./ToDoApp.Api/
RUN dotnet restore ./ToDoApp.Api/ToDoApp.Api.csproj
COPY TodoApp.Api/ ./ToDoApp.Api/
RUN dotnet publish ./ToDoApp.Api/ToDoApp.Api.csproj -c Release -o /publish

# Etape 2 : Runtime (~200MB) pour executer
FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /publish .
EXPOSE 8080
ENTRYPOINT ["dotnet", "ToDoApp.Api.dll"]
```

■ Pourquoi 2 etapes ?

L'image SDK pese ~700MB. L'image runtime ~200MB. En production on ne veut pas le compilateur — juste ce qui execute.

14 Dockerfile frontend + nginx.conf

```
# Etape 1 : Build React
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Etape 2 : Nginx sert les fichiers statiques
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

Dans nginx.conf, le bloc important :

```
location /api/ {
    proxy_pass http://backend:8080;
}
```

15 docker-compose.yml a la racine

```
services:
  backend:
    build: ./backend
    ports: ["5000:8080"]
    volumes: [db-data:/app/data]
```

```
  frontend:
    build: ./frontend
    ports: ["3000:80"]
    depends_on: [backend]
```

```
volumes:
  db-data:
```

```
# Lancer tout
docker-compose up --build
# Frontend : http://localhost:3000
# Swagger : http://localhost:5000/swagger
```

Commit : feat: ajouter Docker et docker-compose

5. Checklist finale

Backend

- ☐ Les 5 endpoints fonctionnent dans Swagger (200/201/204/404)
- ☐ La BDD SQLite se cree automatiquement au demarrage
- ☐ CORS configure — pas d'erreur dans la console du navigateur

Frontend

- ☐ Creer, modifier, cocher, supprimer une tache fonctionne
- ☐ Les filtres "En cours" / "Terminees" fonctionnent
- ☐ La recherche fonctionne
- ☐ Gestion d'erreur si le backend est arrete

Git

- ☐ Des commits propres avec des messages clairs (feat/fix/docs)
- ☐ Le .gitignore ne laisse pas passer bin/, node_modules/, *.db
- ☐ Le repo est pousse sur GitHub

Docker

- ☐ docker-compose up --build lance tout sans erreur
- ☐ http://localhost:3000 fonctionne

Presentation

- ☐ Tu peux expliquer l'architecture en 3 minutes (suivre les schemas)
- ☐ Tu peux ouvrir n'importe quel fichier et expliquer son role
- ☐ Tu as une liste de ce que tu ameliorerais (tests, auth, pagination)